

selenium-bot

Proje Dokümantasyonu

com.ids.bot · Maven / Java 21 / Selenium 4.21.0

IDS / ML Staj Projesi · LIMOS-CNRS, Clermont-Ferrand

Mustafa Göçmen · Abdullah Gül Üniversitesi

Danışman: Gérard Chalhoub · 5 Temmuz 2026

Bu belge Salı toplantısı için hazırlanmıştır. Proje mimarisi, istatistiksel metodoloji, tüm bileşenler, alınan kararlar ve deneysel bulgular detaylıca açıklanmıştır.

İçindekiler

- 1. Projenin Amacı ve Bağlamı**
- 2. Sistem Mimarisi — 3 Makine, 1 Ağ**
- 3. Proje Yapısı ve Katmanlar**
- 4. Giriş Noktaları**
 - 4.1 App.java — Tek Senaryo
 - 4.2 ParallelRunner.java — Tek Seferlik 3 Bot
 - 4.3 MixedTrafficRunner.java — Sürekli Karışık Trafik
- 5. Page Object Katmanı — Selenium Soyutlaması**
 - 5.1 BasePage.java
 - 5.2 HomePage / ProductsPage / ProductDetailPage / ContactPage
- 6. Senaryo Katmanı — Kullanıcı Davranışı**
 - 6.1 BrowsingScenario — Gezici Kullanıcı
 - 6.2 SearchingScenario — Ürün Arayan
 - 6.3 FormFillingScenario — Form Dolduran
 - 6.4 waitBetweenActions() — Evrensel Bekleme Kuralı
- 7. Utility Katmanı — İstatistiksel Motor**
 - 7.1 BotState — Durum Enumu
 - 7.2 MarkovTransitionModel — Zincir Motoru
 - 7.3 BotPersonas — Geçiş Matrisleri
 - 7.4 TimingProfile ve TimingProfiles
 - 7.5 SessionIntensity — Oturum Uzunluğu
 - 7.6 UserAgentPool — Kimlik Sahteciliği
 - 7.7 FormDataPool
- 8. İstatistiksel Metodoloji**
 - 8.1 CICIDS2017 Veri Kaynağı
 - 8.2 Log-Normal Dağılım Modeli
 - 8.3 Zero-Inflated (Sıfır Şişirilmiş) Model
 - 8.4 KMeans Kümeleme ile Persona Ayrımı
 - 8.5 Little's Law ile Varış Hızı Kalibrasyonu
 - 8.6 Markov Olasılıkları — Dürüstlük Notu
- 9. CDP Entegrasyonu — Tarayıcı Kontrol Katmanı**
- 10. Kritik Hatalar ve Düzeltmeleri**
 - 10.1 Cache Devre Dışı — Sıra Kritik
 - 10.2 PRODUCT_DETAIL Navigasyon Hatası
 - 10.3 Alert Race Condition
 - 10.4 Arrival-Capacity Karışıklığı
- 11. Deneysel Bulgular — RITA Beacon Testi**
- 12. Test Yapısı**
- 13. Veri Akışı Uçtan Uca**
- 14. Bilinen Kısıtlamalar**
- 15. Çalıştırma Komutları**

1. Projenin Amacı ve Bağlamı

Bu proje, LIMOS-CNRS'teki IDS (Intrusion Detection System) araştırma stajının Faz 1 çıktısıdır. Temel soru şudur: Geleneksel ağ güvenlik araçları (Zeek, RITA) ile tespit edilemeyen 'akıllı' bot trafiğini, makine öğrenmesi (Autoencoder) tespit edebilir mi?

selenium-bot bu soruyu test etmek için kullanılan kontrollü benign trafik üreticisidir. Headless Chrome ile gerçek bir tarayıcı üzerinden çalışır, CICIDS2017 veri setinden türetilen istatistiksel parametrelerle insan davranışını taklit eder.

Complémentarité Tezi

Projenin ana hipotezi: Klasik kural-tabanlı NIDS (imza eşleştirme, periyodiklik tespiti) ve makine öğrenmesi tabanlı anomali tespiti birbirini tamamlar — ikisi birlikte tek başına her birinden daha güçlüdür. Bu tezi destekleyen kanıt:

- RITA beacon taraması: bot trafiği 0.27-0.64 arasında dağılıyor, 'High' eşliğine girmiyor
- İnsan trafiğinden biri (yavaş gezinme) 0.89 ile bot testlerinin çoğundan yüksek çıktı
- Yani RITA bu trafik türünü net olarak ayırt edemiyor — Autoencoder'ın dolduracağı boşluk
- rare_signature modifier UA çeşitliliğinden bağımsız sinyal üretiyor — ek katman

Faz 1 (tamamlandı)	Benign trafik üreticisi (bu proje) — istatistiksel kalibrasyon
Faz 2 (sıradaki)	Zeek loglarından feature extraction, Autoencoder eğitimi (Pi'de)
Nihai hedef	Autoencoder'ın RITA'nın kaçırdığı örüntüyü tespit etmesi
Test ortamı	Tamamen izole lab ağı — internet trafiği karışmıyor

2. Sistem Mimarisi — 3 Makine, 1 Ağ

Lab ağı 3 fiziksel makineden oluşur. Hepsi Powermaster 8-port unmanaged switch üzerinden ethernet ile bağlı. İnternet erişimi WiFi üzerinden; Zeek sadece eth0'ı (kablolu arayüz) dinler — WiFi trafiği analiz dışı.

Makine	IP (eth0)	Rol	Araçlar
Raspberry Pi 5	192.168.10.1	Kurban web sunucu + ML motoru	Nginx (TechMarket sitesi), Zeek 8.2, RITA v5.1.2, Docker, Autoencoder (Faz 2)
Dell Laptop	192.168.10.2	Saldırgan makinesi	Apache Bench, Slowloris, Python 3.14.6
MacBook Pro M2	192.168.10.3	Benign trafik üreticisi	Locust 2.44.4, selenium-bot (bu proje), Maven 3.9, Python 3

Kritik kural: Bot MacBook'ta çalışır, Pi'deki siteye HTTP isteği gönderir, Zeek Pi'nin eth0'ında bu trafiği loglar. Bot Pi'de çalıştırılırsa trafik loopback'ten geçer ve Zeek görmez — bu proje tarihinde yaşanmış ve kayıt altına alınmış bir hatadır.

TechMarket Test Sitesi

Pi'deki Nginx üzerinde barındırılan 14 sayfalık sahte e-ticaret sitesi. Statik HTML — backend yok. Contact formu POST /submit-contact'a gönderir, Nginx handler olmadığı için 404 döner ama istek loglanır (Zeek görür). 17 görsel, 5-88 KB arası — gerçekçi paket boyutu çeşitliliği için.

DNS: Split-DNS Yapısı

Pi'de dnsmasq: techmarket.lab => 192.168.10.1. MacBook'ta /etc/resolver/lab konfigürasyonu. Chrome'da --disable-features=AsyncDns zorunlu: olmadan Chrome kendi dahili resolver'ını kullanır ve .lab TLD'yi çözemez.

Ölçüm Zinciri

MacBook (selenium-bot) -> ethernet -> Pi (Nginx/Zeek) -> conn.log / http.log / dns.log -> feature extraction (Faz 2) -> Autoencoder

3. Proje Yapısı ve Katmanlar

```
selenium-bot/
├─ pom.xml # Maven: Java 21, Selenium 4.21, JUnit 5.10
├─ src/main/java/com/ids/bot/
│   ├── App.java # Tek senaryo – CLI giriş noktası
│   ├── ParallelRunner.java # 3 bot tek seferlik paralel
│   ├── MixedTrafficRunner.java # Sürekli karışık trafik simülasyonu
│   └─ pages/ # Katman 1: Selenium soyutlaması
│       ├── BasePage.java # Ortak altyapı (explicit wait)
│       ├── HomePage.java
│       ├── ProductsPage.java
│       ├── ProductDetailPage.java
│       └─ ContactPage.java
│   └─ scenarios/ # Katman 2: Kullanıcı davranışı
│       ├── BrowsingScenario.java
│       ├── SearchingScenario.java
│       └─ FormFillingScenario.java
│   └─ util/ # Katman 3: İstatistiksel motor
│       ├── BotState.java # Enum: HOME/PRODUCTS/.../CONTACT
│       ├── MarkovTransitionModel.java # Genel Markov zinciri motoru
│       ├── BotPersonas.java # 3 persona geçiş matrisi
│       ├── TimingProfile.java # Zamanlama parametreleri (record)
│       ├── TimingProfiles.java # Fabrika: browsing/searching/formFilling
│       ├── SessionIntensity.java # Gaussian oturum uzunluğu
│       ├── UserAgentPool.java # 20 Chromium UA girdisi
│       ├── FormDataPool.java # Türkçe form verisi (kasıtlı)
│       └─ CicidsParameterGenerator # @Deprecated – aggregate'e delege
```

4 Bağımsızlık Katmanı

Her katman alttakine bağlıdır, üstekini bilmez. Senaryo sınıfları hangi CSS selector'ın kullanıldığını bilmez. Page sınıfları Markov zincirinin ne olduğunu bilmez. Bu bağımsızlık her katmanı ayrı test edilebilir kılar.

Katman	Sınıflar	Sorumluluk
Giriş	App, ParallelRunner, MixedTrafficRunner	CLI argüman işleme, ChromeDriver fabrikası, iş koordinasyonu
Page Object	BasePage + 4 sayfa sınıfı	Selenium element etkileşimi, explicit wait, selector izolasyonu
Senaryo	3 senaryo sınıfı	Markov navigasyonu + CICIDS zamanlama = kullanıcı davranış simülasyonu
Utility	9 sınıf	Durum makinesi, istatistiksel dağılım, UA havuzu — durum bilgisiz, thread-safe

4. Giriş Noktaları

4.1 App.java — Tek Senaryo

En basit giriş noktası. CLI argümanından senaryo adı alır (browsing / searching / formfilling), yoksa 'browsing' varsayılan. createDriver() statik fabrika metodu App, ParallelRunner ve testler tarafından ortak kullanılır.

createDriver() ne yapar?

- ChromeOptions: --headless, --no-sandbox, --incognito, --disk-cache-size=0
- --disable-features=AsyncDns: Chrome'un dahili DNS resolver'ını devre dışı bırakır (techmarket.lab için şart)
- CDP Network.setUserAgentOverride: UA string + Client Hints metadata override
- CDP Network.enable -> Network.setCacheDisabled(true): cache tamamen kapalı (sıra kritik!)

4.2 ParallelRunner.java — Tek Seferlik 3 Bot

Sabit üç Callable (browsing / searching / formfilling) oluşturur, invokeAll() ile aynı anda başlatır ve hepsinin bitmesini bekler. Her bot kendi izole ChromeDriver'ına sahip, thread-safe. BotResult record: personaName, success, errorMessage.

Not: Kullanım senaryosu: kısa, tek bir tur veri toplamak. Uzun süreli gerçekçi simülasyon için MixedTrafficRunner kullanılır.

4.3 MixedTrafficRunner.java — Sürekli Karışık Trafik

Sabit süre boyunca sürekli yeni bot oturumları başlatan gelişmiş çalıştırıcı. Gerçek web sitesinde farklı zamanlarda gelen bağımsız kullanıcıları modeller. Mimarinin özü: varış süreci ile kapasite kontrolü tamamen ayrılmıştır.

Persona ağırlıkları	Browsing %60 / Searching %30 / FormFilling %10
Varsayılan süre	15 dakika (argümanla override: mvn exec ... -Dexec.args='18')
MAX_CONCURRENT	3 — aynı anda en fazla 3 aktif oturum
Varış aralığı	uniform(28 s, 72 s) — ortalama IAT = 50 s
Executor türü	newCachedThreadPool() — semaphore gerçek kapasite kapısı

Varış-Kapasite Ayrımı — Neden Önemli?

İlk versiyon slots.acquire() çağrısını ana döngüde yapıyordu. Bu şu soruna yol açar: kapasite dolduğunda ana döngü bloklanır, bir sonraki 'kullanıcının gelmesi' gecikir, varış zamanlaması bozulur. Düzeltme: ana döngü hiç bloklanmaz, acquire() task thread'inde yapılır.

```
// Ana döngü — hiç bloklanmaz
while (Instant.now().isBefore(deadline)) {
    Thread.sleep(uniform(28_000, 72_000)); // varış aralığı
    Persona persona = Persona.sampleWeighted();
    int id = launched.incrementAndGet();
```

```
executor.submit(() -> {
    slots.acquire(); // BURADA bekleme — sadece bu task thread'i
    try { runBot(id, persona); }
    finally { slots.release(); }
});
}
```

newCachedThreadPool — Neden newFixedThreadPool Değil?

newFixedThreadPool(3) kullanılsaydı iki çakışan kapasite kısıtı olurdu: pool boyutu (3) VE semaphore (3). Birini değiştirip diğerini unutmak mümkün. newCachedThreadPool ile semaphore tek gerçek kapasite kapısıdır.

Not: newFixedThreadPool deadlock'a YOL AÇMAZ. Kuyruktaki task hiçbir permit tutmadan pasif bekler — circular wait koşulu oluşmaz. Gerekçe doğruluk, deadlock önleme değil.

Little's Law ile Varış Hızı Kalibrasyonu

Little's Law: $L = \lambda \times W$. L = sistemdeki ortalama iş sayısı, λ = varış hızı, W = ortalama hizmet süresi. $\rho = L / \text{kapasite} = \lambda \times W / \text{MAX_CONCURRENT}$.

PRODUCT_DETAIL navigasyon düzeltmesi oturum süresini artırdı. Markov simülasyonu ile ölçülen ağırlıklı ortalama: $W \approx 101.6$ s. Hedef $\rho \approx 0.68$ (kararlı, kuyruk birikmez). Çözüm: IAT uniform(25s, 65s) -> uniform(28s, 72s), $\lambda = 1/50$ s.

Versiyon	IAT	lambda	W	rho	Sonuç
v1 (ilk)	3-15 s	1/9 s	92 s	3.4	Kuyruk sinirsiz buyuyor, zorla kesme
v2	25-65 s	1/45 s	92 s	0.68	Kararlı — PRODUCT_DETAIL fix öncesi
v3 (güncel)	28-72 s	1/50 s	102 s	0.68	Kararlı — güncel kalibrasyon

5. Page Object Katmanı — Selenium Soyutlaması

Her web sayfası bir Java sınıfıyla temsil edilir. Selenium selector'ları (CSS/XPath) ve element etkileşimleri page sınıflarında kapsül edilir. Senaryo sınıfları selector detaylarını bilmez — bu sayede site HTML'i değiştiğinde yalnızca ilgili page sınıfı güncellenir.

5.1 BasePage.java — Temel Altyapı

Tüm page sınıflarının üst sınıfı. İki kritik tasarım kararı:

- Implicit wait ASLA set edilmez — Selenium dökümantasyonu bunu açıkça yasaklar. Implicit + explicit wait karıştırılırsa tahmin edilemez timing davranışları ortaya çıkar.
- waitForElement(By): ExpectedConditions.visibilityOfElementLocated — sadece metin/değer okuma için
- click(By): AYRI WebDriverWait + ExpectedConditions.elementToBeClickable — görünür OLMAK ile tıklanabilir OLMAK farklı kavramlardır

click() ve waitForElement() ayrımı neden önemlidir? visibilityOfElementLocated yalnızca elementin DOM'da görüldüğünü garanti eder. Tıklanabilirlik ayrıca şu koşulları gerektirir: enabled=true, başka bir element tarafından intercept edilmemiş. Paralel yükte (MixedTrafficRunner, n=26) bu fark gerçek bir hataya yol açtı: addToCart() click() başarılı görünüyordu ama JS alert tetiklenmiyordu.

5.2 Sayfa Sınıfları

Sınıf	Temel Metodlar	Notlar
HomePage	open() nav linkleri	http://techmarket.lab/index.html
ProductsPage	open() getProductCount() clickRandomProduct()	getProductCount(): .product-card a, .product-item a, a[href*=product-] selector
ProductDetailPage	getProductName() addToCart()	addToCart(): tıklama + WebDriverWait.until(alertIsPresent()) + accept(). Ag trafiği URETMEZ — sadece JS alert.
ContactPage	fillForm() submit()	submit(): native HTML submit -> gerçek POST /submit-contact HTTP istegi (Zeek gorur). selectByVisibleText() Turkce SUBJECTS gerektirir.

6. Senaryo Katmanı — Kullanıcı Davranışı

Her senaryo bir kullanıcı tipini simüle eder. Navigasyon Markov zinciriyle, zamanlama CICIDS2017 istatistikleriyle belirlenir. Senaryo sınıfları durum bilgisini (current BotState) kendileri yönetir.

6.1 BrowsingScenario — Gezici Kullanıcı

Siteyi yavaşça gezen, ürün detaylarına giren ama sepete eklemek için acele etmeyen kullanıcı. pActiveIdleCycle=0.915 — çoğu bekleme süresi active+idle döngüsünden oluşur, uzun oturum süresi üretir.

SessionIntensity	BROWSING — Gaussian(mean=8, std=2.5), clamp [3..15] adım
TimingProfile	TimingProfiles.browsing() — pActiveIdleCycle=0.915, aktif medyan ~2.8 s
Markov modeli	BotPersonas.browsingModel() — 5 durum arası serbest gezinti
Ağırlıklı W	~120 s (en uzun oturum tipi, BROWSING pActiveIdleCycle yüksek olduğundan)

6.2 SearchingScenario — Ürün Arayan

Belirli bir ürün arayan, çoğunlukla PRODUCTS->PRODUCT_DETAIL->PRODUCTS döngüsünde dolaşan odaklı kullanıcı. pActiveIdleCycle=0.164 — çoğu bekleme zero-inflated forward wait, daha kısa ve öngörülebilir.

SessionIntensity	SEARCHING — Gaussian(mean=5, std=1.8), clamp [2..10] adım
TimingProfile	TimingProfiles.searching() — pActiveIdleCycle=0.164
Markov modeli	searchingModel() — PRODUCTS->PRODUCT_DETAIL %80 dominant
%40 addToCart	Ağ trafiği yok (JS alert), yalnızca tıklama davranışı
CONTACT ziyareti	Sayfa açılır ama form doldurulmaz — kasıtlı

6.3 FormFillingScenario — Form Dolduran

Markov zincirine değil, faz yapısına dayalı senaryo. İki ayrı faz vardır:

- Giriş Fazı: SessionIntensity.FORM_FILLING (mean=3, max=6) kadar adım, HOME/ABOUT/PRODUCTS arasında küçük bir formFillingModel Markov zinciriyle kısa gezinti
- İletişim Fazı: contact.html aç, FormDataPool'dan Türkçe veri çek, fillForm() çağır, 5x(400-1200 ms) typing delay (insan yazma hızı), submit()
- Settle: submit sonrası 2500 ms (POST isteğinin tamamlanması için — waitBetweenActions'dan KASITLI ayrı)
- HOME Dönüşü: %30 olasılıkla + 2500 ms ek bekleme

Not: FormDataPool.SUBJECTS ('Genel Bilgi', 'Sipariş Durumu', 'Teknik Destek', 'Diğer') TechMarket contact.html'deki select option'larıyla birebir eşleşmeli. İngilizce'ye çevrilirse selectByVisibleText() sessizce başarısız olur.

6.4 waitBetweenActions() — Evrensel Bekleme Kuralı

Evrensel kural: her navigasyon aksiyonu -> bir waitBetweenActions() çağırısı. BrowsingScenario'da tanımlanır, SearchingScenario delege eder. TimingProfile'dan örnekleme yapılır:

```
if (profile.shouldHaveActiveIdleCycle()) { // Bernoulli(pActiveIdleCycle)
    Thread.sleep(sampleActiveDurationMs()); // expm1(N(activeLogMean,
    activeLogStd))/1000
    Thread.sleep(sampleIdleDurationMs()); // expm1(N(idleLogMean, idleLogStd))/1000
} else {
    Thread.sleep(sampleFwdWaitMs()); // zero-inflated forward wait
}
```

PRODUCT_DETAIL özel durumu: current==PRODUCTS ise open() atlanır (bot zaten doğru sayfada), dolayısıyla waitBetweenActions() da atlanır. current!=PRODUCTS ise open() çağrılır ve hemen ardından waitBetweenActions() çağrılır — bu sayede kural tutarlı uygulanır.

7. Utility Katmanı — İstatistiksel Motor

7.1 BotState — Durum Enumu

5 mantıksal sayfa durumu: HOME, PRODUCTS, PRODUCT_DETAIL, ABOUT, CONTACT. TechMarket'in 14 HTML dosyası bu 5 duruma eşleştirilmiş. Markov zinciri state uzayını bu enum üzerinden tanımlar.

7.2 MarkovTransitionModel — Birinci-Derece Markov Zinciri Motoru

Markov zinciri nedir? Sistemin bir sonraki durumu, yalnızca mevcut durumuna bağlıdır — geçmişe bakılmaz. Bir kullanıcının products.html'deyken bir sonraki gideceği sayfa, daha önce nerede olduğundan bağımsız belirlenir.

MarkovTransitionModel, Map> alır. Validate: her satırın olasılık toplamı 1.0 olmalı (EPSILON=1e-9 tolerans). nextState(current): ThreadLocalRandom'dan [0,1) sayı çekip kümülatif olasılık üzerinde lineer tarama yapar, seçilen durumu döndürür.

```
double r = ThreadLocalRandom.current().nextDouble();
double cumulative = 0.0;
for (Map.Entry<BotState, Double> e : row.entrySet()) {
    cumulative += e.getValue();
    if (r < cumulative) return e.getKey();
}
return lastEntry; // floating-point güvenlik
```

7.3 BotPersonas — Geçiş Matrisleri

Her persona için Markov matrisi üreten fabrika sınıfı. ÖNEMLİ: Bu olasılıklar CICIDS2017'den türetilmedi — davranışsal varsayıma dayanır. CICIDS veri seti URL geçiş bilgisi içermiyor. RetailRocket ve UCI clickstream veri setleri araştırıldı, TechMarket yapısına uymadı.

browsingModel()

Kaynak	HOME	PRODUCTS	PRODUCT_DETAIL	ABOUT	CONTACT
HOME	—	0.45	0.20	0.20	0.15
PRODUCTS	0.20	—	0.55	0.10	0.15
PRODUCT_DETAIL	0.30	0.40	0.20	—	0.10
ABOUT	0.50	0.30	—	—	0.20
CONTACT	0.60	0.40	—	—	—

searchingModel()

Kaynak	HOME	PRODUCTS	PRODUCT_DETAIL	ABOUT	CONTACT
HOME	—	0.60	0.15	0.10	0.15

PRODUCTS	0.10	—	0.80	—	0.10
PRODUCT_DETAIL	0.35	0.50	0.15	—	—
ABOUT	0.40	0.60	—	—	—
CONTACT	0.70	0.30	—	—	—

7.4 TimingProfile ve TimingProfiles

TimingProfile: Immutable Java record, 12 double alan. Tüm log parametreler $\log_{10}$ (microseconds) uzayında — CICIDS veri birimlerinden doğrudan türetili. Örnek üretiminde $\text{Math.exp}(m(\text{sample}) / 1000.0)$ ile ms'ye çevrilir.

Alan	Açıklama
activeLogMean / activeLogStd	Active periyot log-normal parametreleri ($\log_{10}$ us)
activeClampMin/MaxMs	Active periyot clamp sınırları — uç değerleri kes
idleLogMean / idleLogStd	Idle periyot log-normal parametreleri
idleClampMin/MaxMs	Idle periyot clamp sınırları
pActiveIdleCycle	Active+idle döngüsü olasılığı (Bernoulli kapısı)
fwdNearZeroProb	Zero-inflated zero kütle olasılığı
fwdLogMean / fwdLogStd	Forward wait koşullu log-normal parametreleri

Fabrika Metodu	pActiveIdleCycle	activeLogMean	idleLogMean	fwdLogMean	Kaynak
browsing()	0.915	12.0997	16.4975	14.8389	KMeans Kume 2
searching()	0.164	11.7473	15.9133	12.0586	KMeans Kume 0
formFilling()	0.05	11.7473	15.9133	12.0586	Searching param, farklı pAIC
aggregate()	0.378	11.95	16.20	13.45	@Deprecated — geriye uyum

Not: UYARI: `extract_params.py` (ANALYS-BENIGN) $\log_{10}$ (ms) uzayında çalışıyor. Java sabitleri $\log_{10}$ (us) uzayında. Birimler karıştırılırsa 1000x hata olur.

7.5 SessionIntensity — Oturum Uzunluğu

Gaussian dağılımdan oturum adım sayısı örnekler. Her persona için ayrı istatistik. `sample()` = `clamp(round(N(mean, std)), min, max)`.

Enum	mean	std	min	max	Senaryo
BROWSING	8.0	2.5	3	15	BrowsingScenario — uzun gezinti
SEARCHING	5.0	1.8	2	10	SearchingScenario — odaklı arama
FORM_FILLING	3.0	1.2	1	6	FormFillingScenario giriş fazı — kısa

7.6 UserAgentPool — Kimlik Sahteciliği

20 girişli statik UA havuzu. Her giriş UserAgentEntry record: UA string, platform, platformVersion, arch, model, mobile, brandName, majorVersion, fullVersion.

- Yalnızca Chromium ailesi: Chrome x14, Edge x6 — Firefox/Safari/iOS ASLA eklenmemeli
- Neden? Gerçek istemci headless Chrome. sec-ch-ua Client Hints imzası her zaman Chromium. Farklı UA seçilirse fingerprint tutarsızlığı — tespit edilebilir anomali

-
- OS çeşitliliği: Windows 10, Windows 11, macOS Intel, macOS ARM, Linux, Android
 - Windows 11 bug: UA string 'NT 10.0' yazar, Win11 platformVersion='15.0.0' ile ayrılır
 - buildMetadata(): GREASE token + Chromium + marka brands listesi + fullVersionList (yüksek-entropi hint)

7.7 FormDataPool

İletişim formu için Türkçe sahte veri. NAMES(10), EMAILS(10), SUBJECTS(4), MESSAGES(5). SUBJECTS: 'Genel Bilgi', 'Sipariş Durumu', 'Teknik Destek', 'Diğer'. Bu değerler TechMarket HTML'indeki select option'larıyla birebir eşleşmeli. İngilizce'ye çevrilirse selectByVisibleText() sessizce başarısız olur.

8. İstatistiksel Metodoloji

8.1 CICIDS2017 Veri Kaynağı

CICIDS2017 Monday-WorkingHours BENIGN trafiği: 529.918 akış, 437 Inf/NaN + 14 sentinel temizlendi, 191.870 HTTP/HTTPS akışı analiz edildi. CICFlowMeter çıktısı — birimler mikrosaniye.

Önemli: Bu veri 2017'ye ait ve gerçek bir ofis ağından. TechMarket trafiğimiz kalibrasyonun ruhunu alıyor — sayısal değerlerin bire bir uyması beklenmez, istatistiksel dağılım şeklinin korunması hedeflenir.

8.2 Log-Normal Dağılım Modeli

Problem: Ağ akış süreleri ve boyutları yüksek çarpıklık gösterir (skewness 20-225). Normal dağılım kullanılamaz — çoğu değer küçük, az sayıda değer çok büyük. Çözüm: log-normal dağılım.

$\log_{10}(\text{raw_us})$ alınır -> log uzayında normal dağılım uygulanır -> $\expm1(\text{sample}) / 1000.0$ ile ms'ye çevrilir. $\log_{10}$ kullanılır (log yerine) çünkü sıfır değerler var ve $\log(0) = -\infty$.

```
// Ornekleme
double logSample = logMean + logStd * ThreadLocalRandom.current().nextGaussian();
double ms = Math.expml(logSample) / 1000.0;

// Tersine çevirme (parametreler nasıl hesaplandı)
double log_value = Math.log1p(raw_microseconds);
// Bu log_value'ların ortalama/std'si -> logMean / logStd
```

8.3 Zero-Inflated (Sıfır Şişirilmiş) Model

CICIDS verisi incelendiğinde: akışların %67'sinde aktif/idle periyot sıfır. Yani büyük çoğunluk doğrudan forward wait yaşıyor. Tek bir log-normal ile modellenemez — zero-inflation var.

Çözüm: Bernoulli kapısı + koşullu log-normal. $p_{\text{ActiveIdleCycle}} = P(\text{active+idle döngüsü gerçekleşir})$. Bu değer KMeans kümelemesinden çıkar: Küme 0 (Searching) %16.4, Küme 2 (Browsing) %91.5.

```
if (rand.nextDouble() < pActiveIdleCycle) {
    // Active+idle döngüsü var
    sleep(clamp(expml(N(activeLogMean, activeLogStd))/1000, min, max));
    sleep(clamp(expml(N(idleLogMean, idleLogStd))/1000, min, max));
} else {
    // Zero-inflated forward wait
    if (rand.nextDouble() < fwdNearZeroProb)
        sleep(uniform(1, 50)); // near-zero kütle
    else
        sleep(rejectionSample()); // log-normal, 100 deneme limiti
}
```

8.4 KMeans Kümeleme ile Persona Ayrımı

191.691 akış üzerinde KMeans (k=2). Silhouette skoru: k=2 için 0.556, k=3 için 0.453. Verinin doğal yapısı 2 küme — daha fazla persona istatistiksel olarak desteklenmiyor.

- Küme 0 (Searching): Odaklı, kısa active periyotlar, $p_{\text{ActiveIdleCycle}} = 16.4$

-
- Küme 2 (Browsing): Aylak, uzun active/idle, pActiveIdleCycle=%91.5
 - Asset fetch akışları (~45k, near-zero): insan davranışı değil, Chrome'un otomatik yüklemeleri — ayrıca modellenmez
 - FormFilling: Searching parametreleri + pActiveIdleCycle=0.05 (faz yapısı ayırt eder)

8.5 Little's Law ile Varış Hızı Kalibrasyonu

Önceki bölümde (4.3) detaylıca açıklandı. Özet: $W \approx 102$ s, $\lambda = 1/50$ s, $L = \lambda * W \approx 2.04$, $\rho = L/3 \approx 0.68$. $\rho < 1$ olmak kararlılık garantisi. $\rho > 1$ olduğunda kuyruk teorik olarak sonsuza gider, pratikte `executor.shutdownNow()` oturumları kesiyor — istenmeyen durum.

8.6 Markov Olasılıkları — Dürüstlük Notu

`BotPersonas.java`'daki geçiş olasılıkları CICIDS2017'den türetilmedi. Davranışsal varsayıma dayanır. Bu durum kodun javadoc'unda açıkça belirtilmiştir. Akademik çalışmalarda bu ayrımı net tutmak önemlidir.

9. CDP Entegrasyonu — Tarayıcı Kontrol Katmanı

CDP (Chrome DevTools Protocol): Chrome tarayıcısını programatik olarak kontrol etmeye yarayan düşük seviye protokol. Selenium normal olarak WebDriver API'sini kullanır; CDP ile bunun ötesine geçilir.

Neden Untyped executeCdpCommand?

Selenium'un typed CDP binding'leri (org.openqa.selenium.devtools.v125...) yalnızca Chrome v125'e kadar desteklenir. Daha yeni Chrome'da 'Unable to find CDP implementation' uyarısı çıkar ve API kırılır. Untyped executeCdpCommand tüm Chrome sürümlerinde çalışır.

Network.setUserAgentOverride

UA string'ini ve Client Hints metadata'sını aynı anda override eder. Parametreler: userAgent, platform, userAgentMetadata (brands, fullVersionList, mobile, architecture, bitness...). sec-ch-ua header'larını da etkiler — yalnızca UA string'i değil, tüm fingerprint tutarlı hale gelir.

Network.enable -> Network.setCacheDisabled(true)

Cache kapatılmazsa tekrar ziyaret edilen sayfalar Chrome cache'inden gelir, ağ istekleri oluşmaz, Zeek görmez — Faz 2 feature extraction için kritik veri bütünlüğü sorunu.

UYARI: Network.enable çağrılmadan Network.setCacheDisabled sessizce no-op kalır. Bu sıra hatası proje tarihinde yaşandı ve Zeek loglarındaki eksik kayıtlarla tespit edildi.

10. Kritik Hatalar ve Düzeltmeleri

Bu bölüm proje süresince keşfedilen ve düzeltilen gerçek hataları belgeler. Her biri bir şeyin neden böyle tasarlandığını açıklar.

10.1 Cache Devre Dışı — Sıra Kritik

Hata: `Network.setCacheDisabled` önce, `Network.enable` sonra çağrıldı. Cache kapatılmadı, sayfa yeniden ziyaretleri Zeek'te görünmedi.

Düzeltilme: `Network.enable` -> `Network.setCacheDisabled` sırası zorlandı. Comment: 'ORDER MATTERS: `setCacheDisabled` silently no-ops unless `Network.enable` is called first.'

10.2 PRODUCT_DETAIL Navigasyon Hatası

Sorun: `BrowsingScenario/SearchingScenario`, `PRODUCT_DETAIL` durumuna geçerken her seferinde koşulsuz `productsPage.open()` çağırıyordu.

Sonuç: `PRODUCTS`->`PRODUCT_DETAIL` (%55/%80 dominant geçiş) durumunda bot zaten `products.html`'deyken sayfayı gereksizce yeniden yüklüyor. İki HTTP isteği (reload + tıklama) tek `waitBetweenActions()`'a sıkışıyor. `action_level_comparison.py`'de browsing medyan 6.5s/13.8s sapması bu nedenle.

Düzeltilme 1: `if (current != BotState.PRODUCTS) productsPage.open();` — dominant geçişte gereksiz reload kaldırıldı.

Düzeltilme 2: `open()` çağrıldığında hemen ardından `waitBetweenActions()` eklendi — evrensel kural tutarlı uygulandı (1 navigasyon = 1 wait).

KS testi sonucu: browsing p-value 0.0000 -> 0.4442 (tutarlı), searching 0.0039 -> 0.2470 (tutarlı). N=5 doğrulama.

10.3 Alert Race Condition

Hata: `ProductDetailPage.addToCart()` `click()` ardından `driver.switchTo().alert()` wait olmadan çağırıyordu. Sequential testlerde sorun çıkmıyordu.

MixedTrafficRunner altında paralel yük (n=26 koşu) bu durumu gün yüzüne çıkardı: 1 kez 'waiting for alert to be present' timeout — `click()` başarılıydı ama JS alert henüz gösterilmemişti.

Düzeltilme: `WebDriverWait + ExpectedConditions.alertIsPresent()` eklendi. try/catch ile yutmak yerine doğru bekleme — sessiz veri kaybı önendi.

10.4 Arrival-Capacity Karışıklığı (MixedTrafficRunner)

Hata: `slots.acquire()` ana döngüde yapılıyordu. Kapasite dolduğunda ana döngü bloklanıyor, varış zamanlaması bozuluyordu. $\rho=3.4$ ile kuyruk sınırsız büyüdü, 10 dk'lık koşu 20 dk sürüp `shutdownNow()` ile oturumlar zorla kesildi.

Düzeltilme: `slots.acquire()` task thread'ine taşındı. Ana döngü hiç bloklanmaz. IAT 25-65 s -> 28-72 s (W artışı için), $\rho=0.68$.

11. Deneyisel Bulgular — RITA Beacon Testi

RITA (Real Intelligence Threat Analytics) v5.1.2, Zeek loglarında beacon davranışı arar: düşük varyanslı, periyodik bağlantı deseni. C2 (Command & Control) trafiğine tipik olarak bu imza bulunur.

Test Kurulumu

Lab ağı tamamen 192.168.0.0/16 içinde. Varsayılan RITA ayarı internal-to-internal trafiği filtreler. Çözüm: `always_included_subnets` direktifi ile `internal_subnets` override edildi — bot trafiği analize dahil oldu.

Tam Test Sonuçları

Test Koşulları	Beacon Score	RITA Severity	Yorum
Genel ölçüm (sabah)	0.275	None	Karma trafik, düşük periyodiklik
Genel ölçüm (akşam, kirli veri)	0.426	None	Eski pcap karışımı — sonuç geçersiz
Bot, 3-eşzamanlı (MixedTrafficRunner)	0.638	Low	En yüksek bot skoru
Bot, tek-sıralı	0.568	Low	Düşük eşzamanlılık
Bot, 7-bağlantı örnekleme	0.615	Low	Küçük örneklem
İnsan, hızlı gezinme	0.272	None	Karışık gezinme = düşük periyodiklik
İnsan, yavaş gezinme	0.892	Medium	En yüksek skor — BOT'tan yüksek!

Bulgunun Yorumu

Sonuç: 0.27 ile 0.89 arasında dağılmış, hiçbir test 'High' (0.9+) kategorisine girmedir. Daha da önemlisi: yavaş gezen bir insan kullanıcısı (0.892), tüm bot testlerinden daha yüksek skor aldı.

Bu ne anlama gelir? RITA bu trafik türünü net olarak ayırt edemiyor. Ne kesin 'zararsız' ne kesin 'şüpheli' diyor. Peki neden? Çünkü log-normal zamanlama + Markov navigasyonu düşük varyanslı periyodik imza üretmiyor — klasik C2 beacon'larından istatistiksel olarak farklı.

Bu bulgu projenin 'complémentarité' tezini doğrudan destekliyor: kural-tabanlı NIDS yetersiz kalıyor, Autoencoder'ın dolduracağı açık bir boşluk var. Faz 2'nin motivasyonu buradan geliyor.

rare_signature Modifier

RITA'nın `rare_signature` modifier'ı, az görülen User-Agent string'lerini flaglar. UA havuzumuzdaki 20 farklı Chromium girdisi, tekrarlayan oturumlarda farklı UA'lar kullanarak bu sinyali karıştırıyor. Beacon skoru mekanizmasından bağımsız ek bir katman — çok-katmanlı savunma tezine ek kanıt.

Teknik Not: Zeek conn.log Duration Tutarsızlığı

Test sırasında Zeek `conn.log`'daki `duration` alanının, ham tcpdump'ta gözlenen TCP açık-kalma süreleriyle uyuşmadığı fark edildi. Checksum offloading ile ilişkili olabilir (NIC donanım offloading'i Zeek'in bazı paket

meta verilerini eksik görmesine yol açabilir). Bu detay ayrıca not edildi ama skorların 'kararsız' genel sonucunu deęiřtirmiyor ve üzerine bulgular inşa edilmedi.

12. Test Yapısı

JUnit 5.10.2, iki katman. Unit testler herhangi bir makinede, herhangi bir zamanda saniyeler içinde çalışır. Entegrasyon testleri lab ağı + Chrome gerektirir.

Unit Testler (Lab Gerektirmez)

Test	N	Ne Doğrular
BrowsingScenarioTest sessionLengthWithinExpectedRange	10.000	BROWSING [3..15] aralığını hiç aşmaz
BrowsingScenarioTest markovTransitionsNeverProduceUndeclaredState	20.000	Sadece tanımlı hedeflere gecilir; her kenar en az 1 kez tetiklenir
SearchingScenarioTest sessionLengthWithinExpectedRange	10.000	SEARCHING [2..10] aralığında
SearchingScenarioTest markovTransitionsNeverProduceUndeclaredState	20.000	searchingModel için aynı geçerlilik
FormFillingScenarioTest entryCountWithinExpectedRange	10.000	FORM_FILLING [1..6] aralığında
FormFillingScenarioTest homeReturnRateMatchesTarget	100.000	%30 ± 0.02 HOME dönüş oranı

Entegrasyon Testleri (@Tag('integration'))

PageObjectSmokeTest, BrowsingScenarioTest (lab), SearchingScenarioTest (lab), FormFillingScenarioTest (lab). Lab gereklidir: mvn test -Dgroups=integration

Not: mvn test -DexcludedGroups=integration -> 6/6 geçti, ~1 sn, lab gerektirmez.

13. Veri Akışı Uçtan Uca

```
MixedTrafficRunner.main(args=['18'])

|
+-- Arrival loop (28-72s arasi bekleme)
| Persona = sampleWeighted() [Browsing %60 / Searching %30 / Form %10]
| executor.submit(task)
|
+-- [task thread - slots.acquire() ile kapasite bekle]
App.createDriver(UserAgentPool.randomEntry())
+--> ChromeOptions: headless, incognito, AsyncDns kapali
+--> CDP: setUserAgentOverride (UA + Client Hints)
+--> CDP: Network.enable -> setCacheDisabled(true)
|
+-- BrowsingScenario.run() / SearchingScenario.run() / FormFillingScenario.run()
|
+-- homePage.open() --> HTTP GET /index.html
+-- MarkovModel.nextState(current) --> next sayfa
+-- [PRODUCT_DETAIL icin]
| if current != PRODUCTS: productsPage.open() -> waitBetweenActions()
| clickRandomProduct() --> HTTP GET /product-N.html
+-- waitBetweenActions(timingProfile)
| zero-inflated log-normal ornekleme
+-- tekrar (SessionIntensity.sample() kez)

Her HTTP istegi -> Nginx (Pi) -> Zeek eth0 yakalama
-> conn.log (baglanti suresi, byte sayisi)
-> http.log (URL, method, status, user-agent)
-> dns.log (techmarket.lab sorgusu)

Zeek loglari -> [Faz 2] feature extraction -> Autoencoder eğitimi
```

14. Bilinen Kısıtlamalar

Kısıtlama	Açıklama	Durum
TLS/JA3 fingerprint	Tüm oturumlar aynı headless Chrome'dan — TLS fingerprint tekdüze. HTTP (şifresiz) kullandığımızdan şu an etkisiz.	Bilinçli bırakıldı
Sayfa içi etkileşim	Scroll, mouse hareketi, hover yok. Zeek ağ katmanında çalışır — bu davranışlar HTTP trafiğine yansımaz.	Kapsam dışı
Markov olasılıkları	Davranışsal varsayıma dayalı, CICIDS'ten türetilmedi.	Javadoc'ta belgelendi
FormFilling timing	SUBMIT_SETTLE ve typing delay waitBetweenActions()'dan kasıtlı ayrı.	Kasıtlı tasarım
Rejection sampling	sampleFwdWaitMs() 100 denemede 500ms fallback. Pratikte tetiklenmiyor.	Güvenlik ağı
newCachedThreadPool	Teorik olarak sınırsız thread üretebilir. rho=0.68 ile pratikte sorun yok.	Javadoc'ta belgelendi
Pi'de çalıştırma	Bot Pi'de çalışmamalı — loopback trafiği Zeek'ten kacir.	Ders olarak kayıtlı
Zeek duration	conn.log duration alanı TCP gerçek süre ile uyusmuyor — checksum offloading ilişkili olabilir.	Not alındı, incelenmedi

15. Çalıştırma Komutları

Derleme ve Test

```
cd ~/Desktop/selenium-bot

# Derle
mvn compile

# Hizli unit testler – lab gerektirmez, ~1 sn
mvn test -DexcludedGroups=integration

# Lab entegrasyon testleri
mvn test -Dgroups=integration
```

Tek Senaryo

```
mvn exec:java # varsayılan: browsing
mvn exec:java -Dexec.args='browsing'
mvn exec:java -Dexec.args='searching'
mvn exec:java -Dexec.args='formfilling'
```

Paralel — 3 Bot Tek Seferlik

```
mvn exec:java -Dexec.mainClass=com.ids.bot.ParallelRunner
```

Gerçekçi Karışık Trafik — MixedTrafficRunner

```
# 15 dakika (varsayılan)
mvn exec:java -Dexec.mainClass=com.ids.bot.MixedTrafficRunner

# 18 dakika
mvn exec:java -Dexec.mainClass=com.ids.bot.MixedTrafficRunner -Dexec.args='18'
```

Veri Toplama — capture_multiple_runs.sh

```
# Pi'ye SSH erişilebilir olmalı, sudoers kuralı aktif olmalı
# tcpdump -Z raspberrypi - dosya raspberrypi sahipliğinde oluşur
cd ~/Desktop
bash IDS-Analysis/capture_multiple_runs.sh 5 eth0 192.168.10.1
```

Analiz — action_level_comparison.py

```
cd ~/Desktop/IDS-Analysis

# ONEMLİ: önceki run'ların pcap'leri temizlensin
python3 ANALYS-BENIGN/action_level_comparison.py \
--captures-dir ./captures \
```

```
--personas browsing searching formfilling
```

com.ids.bot selenium-bot · Mustafa Göçmen · Abdullah Gül Üniversitesi / LIMOS-CNRS · 5 Temmuz 2026